```python
import numpy as np
import pandas as pd
from IPython.display import display, Math

from mtx_utils import load_mtx

eps = 1e-3
A, b = load_mtx("tasks/1_4.txt")
n = len(b)

A, b
```

```
(array([[-6.,  6., -8.],
        [ 6., -4.,  9.],
        [-8.,  9., -2.]]),
 array([0., 0., 0.]))
```

Метод вращения

```python
def get_t(a_k_lower: np.ndarray):
    return np.sqrt(np.sum(a_k_lower ** 2))
```

```python
Ak = A.copy()
a_k_lower = np.tril(Ak, k=-1)
iterations = 0
fi_k = 0

V = np.eye(n)

while get_t(a_k_lower) > eps:
    iterations += 1

    i, j = np.unravel_index(np.argmax(np.abs(a_k_lower)), a_k_lower.shape)
    fi_k = 0.5 * np.arctan2(2 * Ak[i, j], Ak[i, i] - Ak[j, j])

    Uk = np.eye(n)
    Uk[i, i] = Uk[j, j] = np.cos(fi_k)
    Uk[i, j] = -np.sin(fi_k)
    Uk[j, i] = np.sin(fi_k)

    Ak = Uk.T @ Ak @ Uk
    V = V @ Uk

    a_k_lower = np.tril(Ak, k=-1)
```

```python
lambdas = np.diag(Ak)

for i, lam in enumerate(lambdas, start=1):
    display(Math(fr"\lambda_{{{i}}} = {lam:.3f}"))

print(f"Итераций: {iterations}")
```

$\lambda_1 = -19.344$

$\lambda_2 = 0.771$

$\lambda_3 = 6.573$

```
Итераций: 6
```

```
for i in range(n):
    vec = V[:, i]
    vec_str = r" \\ ".join(f"{x:.3f}" for x in vec)
    display(Math(fr"x^{{({i+1})}} = \begin{{pmatrix}} {vec_str} \end{{pmatrix}}"))
    print()
```

$$x^{(1)} = \begin{pmatrix} 0.596 \\ -0.567 \\ 0.569 \end{pmatrix}$$

$$x^{(2)} = \begin{pmatrix} 0.762 \\ 0.623 \\ -0.178 \end{pmatrix}$$

$$x^{(3)} = \begin{pmatrix} -0.253 \\ 0.540 \\ 0.803 \end{pmatrix}$$

Проверка

```
eigenvalues, eigenvectors = np.linalg.eig(A)

idx = np.argsort(eigenvalues)
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]
```

```
idx = np.argsort(lambdas)
```

```
display(pd.DataFrame(lambdas).T)
```

	0	1	2
0	-19.344142	0.770697	6.573446

```
lambdas = lambdas[idx]
np.allclose(lambdas, eigenvalues, eps)
```

```
True
```

```
V[:, idx]
```

```
array([[ 0.59591222,  0.76204662, -0.25332505],
       [-0.56672685,  0.62257015,  0.5396546 ],
       [ 0.56895458, -0.17802067,  0.80286944]])
```

```
V = V[:, idx]
np.allclose(A @ V, lambdas * V, eps)
```

```
True
```